

# Day 03

---

Regression with `scikit-learn`

MSU REU Machine Learning Short Course

# Classification vs Regression

---

Same pipeline. Different output.

|                   | Classification          | Regression                  |
|-------------------|-------------------------|-----------------------------|
| <b>Output</b>     | A category (STAR / QSO) | A number (z-band magnitude) |
| <b>Loss</b>       | Misclassification rate  | Mean Squared Error          |
| <b>Evaluation</b> | Confusion matrix, F1    | MSE, $R^2$ , residuals      |
| <b>Example</b>    | "Is this a star?"       | "How bright is this star?"  |

# Linear Regression

---

The model:  $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$

The coefficients  $\beta$  are learned from training data by minimizing MSE.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Same API as KNN. That's the point of scikit-learn.

# How good is your fit? MSE and $R^2$

---

**MSE** — average squared distance between predicted and actual values. Lower is better.

**$R^2$**  — fraction of variance explained by the model. 1.0 is perfect; 0.0 is no better than predicting the mean.

```
from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

$R^2 = 0.95$  sounds great. But always look at the plots.

# Always visualize — Anscombe's Quartet

---

Four datasets. Identical means, variances, correlations, and  $R^2$  values. Completely different patterns.

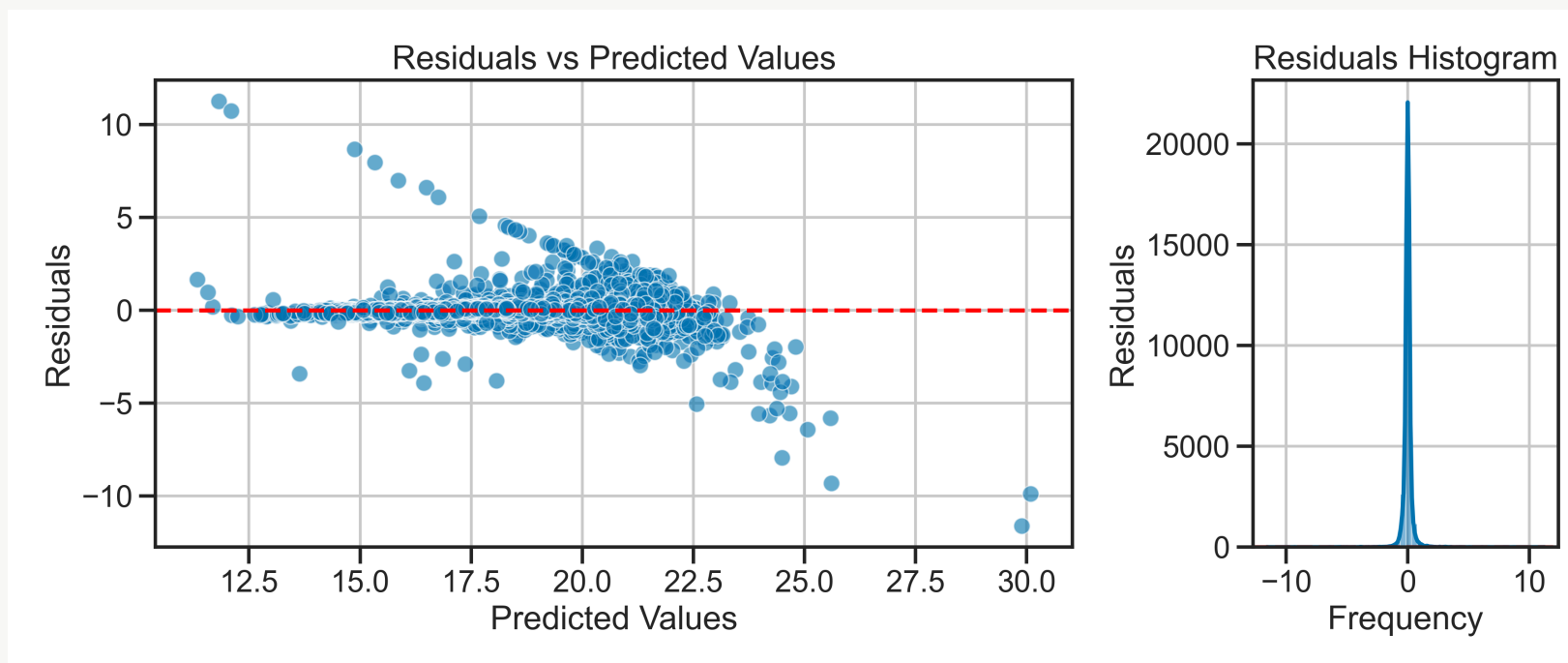
Identical statistics **do not** mean identical data.

See: [The Importance of Visualization](#)

**The lesson:** before trusting any metric, make the plots.

## Three diagnostic plots

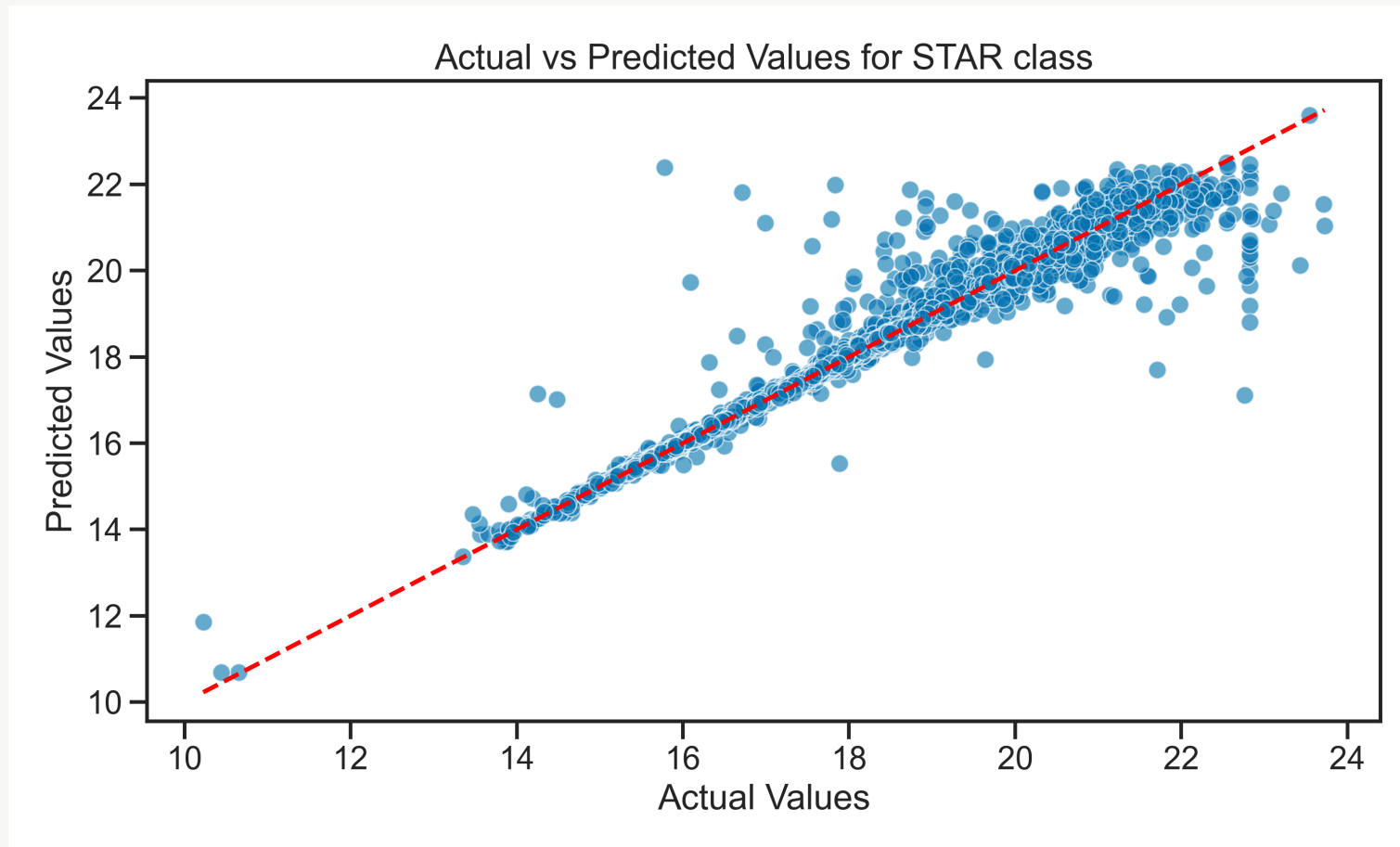
Always make these: residuals, predicted vs actual, and the fit itself.



Residuals should be centered on zero with no structure. If they fan out or curve — your model is missing something.

## When linear isn't enough — Random Forest

Same data, different model. The sklearn API doesn't change.



```
from sklearn.ensemble import RandomForestRegressor
```

# Today's Activity

---

Work through **Activity 03: Regression with scikit-learn**

1. Predict z-band magnitude from other photometric bands
2. Evaluate with MSE,  $R^2$ , and diagnostic plots
3. Add redshift — does it help regression the way it helped classification?
4. Try per-class models (STAR, GALAXY, QSO separately)
5. Compare Linear Regression vs Random Forest

**Note:** [The Importance of Visualization](#)